

Trade Publishing / Reporting classes

Module Goal: Record trades and make them available to consumers (websocket, logs, etc)

{NBBO}: Nation Best bid and Offer

Bid → highest price someone is willing to buy

Offer (Ask) → lowest price someone is willing to sell

Estruct 3

Trade

```
+ double Price
+ double qty
+ std::chrono::system_clock::time_point timestamp
+ uint64_t makerOrderId
+ uint64_t takerOrderId
+ std::string symbol
+ Side takerside
```

Captures full match details

TradePublisher

```
- std::vector<std::function<void(const Trade&)>> subscribers
- std::mutex subMutex
```

```
+ void PublishTrade (const Trade& trade)
+ void subscribe (std::function<void(const Trade&)> callback)
```

- Supports multiple consumers (WebSocket, file, etc)
- Thread-Safe publish/subscribe

Observer pattern
MatchEngine publishes
to it

OrderBook

MatchEngine

Depends on
OrderBook for
real NBBO per
symbol

The system calculates
the last traded price
and the best buy
and sell prices for
each symbol/stock

MarketDataAggregator

```
- std::unordered_map<std::string, double> lastTradePrices
- std::unordered_map<std::string, std::pair<double, double>> nbbo
- std::mutex aggMutex
```

```
+ void updateFromTrade (const Trade& trade)
+ double getLastTradePrice (const std::string& symbol)
+ std::pair<double, double> getNBBO (const std::string& symbol)
```